

SENTINEL

Financial Risk Intelligence Platform

Fraud & Anomaly Detection Platform

An internal transaction monitoring and investigation system that scores every transaction against the customer's own established behaviour, explains each score in full, and carries the resulting investigation through to a recorded outcome.

24,061

LINES OF CODE

48

API ENDPOINTS

265

TESTS PASSING

16

DATABASE TABLES

Technical Project Report · 23 August 2026

Live application sentinel-finance-web.onrender.com

Source github.com/athex911911/sentinel-finance

01 Overview

What the system is, and the principle it is built around.

Sentinel Finance is a decision-support platform for financial crime analysts. It ingests transaction data, engineers behavioural features from each customer's own history, scores every transaction for risk, and gives the analyst a complete, itemised explanation of why a score is what it is. Flagged activity flows into a case management workflow with evidence, notes and an immutable timeline.

THE GOVERNING PRINCIPLE

The system never asserts that fraud occurred. A risk score means a transaction **requires review**. Every number shown to an analyst is traceable to a named rule that actually fired, and the machine learning model's influence is deliberately bounded so that a decision affecting a customer can always be justified in plain language.

Design commitments

COMMITMENT	HOW IT IS ENFORCED IN THE BUILD
Explainability	The displayed score is exactly the sum of the displayed factors. Enforced by an assertion in the scoring engine and by a randomised property test.
Honesty about ML	Precision, recall and confusion matrices are not shown, because no labelled fraud ground truth exists. The model page states this rather than fabricating figures.
No lookahead	Every feature is computed strictly from history preceding its transaction, so a backfilled score equals the score it would have received live. Verified by test.
Auditability	The audit trail is append-only at the database level, not by convention. UPDATE and DELETE are refused by PostgreSQL rules.
Privacy	Customer identifiers are masked in the serialisation layer, so no endpoint can leak them by omission.
Synthetic data only	All demonstration data is generated. No real person, account, device or payment is represented anywhere.

Live deployment

Application	sentinel-finance-web.onrender.com
API	sentinel-finance-api-u5e5.onrender.com
Source repository	github.com/athex911911/sentinel-finance
Transactions under monitoring	12,115
Customers profiled	90
Alerts raised	90
Investigation cases	28

02 Technology Stack

Every dependency, with the reason it was chosen.

Backend — Python 3.11

PACKAGE	VERSION	ROLE AND RATIONALE
FastAPI	0.115.6	HTTP framework. Chosen for native Pydantic integration and automatic OpenAPI generation — the API documentation cannot drift from the code.
Uvicorn	0.34.0	ASGI server running the application in production.
Pydantic	2.10.5	Request validation and response serialisation. PII masking is implemented here so it cannot be bypassed.
pydantic-settings	2.7.1	Typed configuration from environment variables; no secret is ever hardcoded.
SQLAlchemy	2.0.36	ORM and query builder. Parameterised throughout, which structurally prevents SQL injection.
Alembic	1.14.0	Schema migrations, including the raw SQL that installs the append-only audit rules.
psycopg	3.2.3	PostgreSQL driver (version 3, binary distribution).
scikit-learn	1.6.1	Isolation Forest implementation and the StandardScaler feeding it.
pandas	2.2.3	Vectorised feature engineering across customer-partitioned, time-sorted data.
NumPy	2.2.1	Numerical operations, including exact time-window counts via searchsorted .
joblib	1.4.2	Serialises the fitted scaler and model together as one artifact.
PyJWT	2.10.1	Token issuing and verification (HS256).
bcrypt	4.2.1	Password hashing at cost 12. Used directly rather than through passlib, which mis-detects bcrypt 4.x.
openpyxl	3.1.5	Excel export generation.
pytest / httpx	8.3.4 / 0.28.1	Test framework and the client used to exercise the API in tests.

Frontend — TypeScript

PACKAGE	VERSION	ROLE AND RATIONALE
React	19.2	UI library. Default output escaping gives XSS safety without extra work.
TypeScript	6.0	Static typing. API response shapes are typed to mirror the backend schemas.
Vite	8.2	Build tool and dev server. Produces the hashed, code-split production bundle.
TanStack Query	5.101	Server-state management: caching, background refetch, and the interval polling behind the alert centre.
React Router	7.18	Client-side routing across sixteen routes, with lazy-loaded page bundles.
Tailwind CSS	4.3	Styling via a single design-token layer, so risk colours are identical everywhere.
Recharts	3.10	Charting, wrapped in project components that enforce one consistent set of chart decisions.
Lucide React	1.33	Icon set. Icons pair with risk colours so meaning never rests on colour alone.
date-fns / clsx / tailwind-merge	—	Date handling and conditional class composition.

Data and infrastructure

COMPONENT	TECHNOLOGY	NOTES
Database	PostgreSQL 17	Hosted on Supabase. Chosen for native enum types, JSONB, and the rule system that makes the audit trail immutable.
API hosting	Render	Free Python web service, deployed from GitHub via a render.yaml Blueprint.
Frontend hosting	Render Static	CDN-served static bundle with an SPA rewrite rule.
Version control	Git / GitHub	Public repository; secrets excluded and verified absent from the entire history.

03 System Architecture

Strict layering, with each layer talking only to the one beneath it.

api/ — HTTP boundary

Parses requests, authorises, delegates, serialises. Contains no business logic. 14 route modules.



services/ — business logic

Risk scoring, case lifecycle, analytics, customer profiling, import, export, alerts, audit, masking.



repositories/ — querying

All filtering, sorting and pagination expressed in SQL, so the monitoring table scales with the dataset.



db/models/ — persistence

SQLAlchemy ORM across 16 tables with foreign keys, indexes and check constraints.

WHY THE SCORING ENGINE IS ISOLATED

`risk_scoring_service.py` has no knowledge of HTTP or the database. It takes a feature snapshot and returns an assessment. That makes it directly unit testable, and guarantees a transaction is scored identically whether the call arrives through the API, a CSV import, or a batch backfill.

Separation of the three transaction tables

TABLE	PURPOSE
<code>transactions</code>	The immutable financial record exactly as ingested. Never rewritten by detection.
<code>transaction_features</code>	Engineered behavioural signals. Fully recomputable from the source record.
<code>risk_scores</code>	The scoring verdict, versioned against the model that produced it. Also recomputable.

Keeping these apart means re-running detection rewrites features and scores without ever touching the underlying financial record — an audit requirement, not a convenience.

Frontend structure

DIRECTORY	CONTENTS
lib/	Typed API client with automatic token refresh, formatters, and the single risk-presentation source of truth.
components/ui/	Primitives: buttons, inputs, tables, dialogs, skeletons, tooltips, pagination, empty and error states.
components/risk/	Risk badge, score gauge, factor breakdown, demo-data banner, decision-support notice.
charts/	Recharts wrappers carrying one shared set of chart rules — thin marks, recessive grids, hover layer by default.
pages/	Sixteen route components, each lazily loaded to keep the initial bundle small.

04 Database Design

16 tables · 63 indexes · 25 foreign keys · 11 native enum types.

TABLE	PURPOSE	KEY TRAIT
roles	The four access tiers, with privilege ordering stored as an integer level.	Seeded
users	Platform operators. Stores a bcrypt digest only.	Unique email
customers	Account holders under monitoring, with home location and behavioural segment.	Masked out
devices	Devices seen on an account; first_seen_at drives the new-device rule.	Per customer
beneficiaries	Payees an account has transacted with.	Per customer
transactions	The financial record. 11 indexes including a composite on customer and time.	Immutable
transaction_features	24 engineered feature columns, one row per transaction.	Recomputable
risk_scores	Business score, band, ML score, and the factor breakdown as JSONB.	Versioned
alerts	Raised when a score crosses the alert threshold. One per transaction.	Deduplicated
cases	Investigations, with status, priority, assignment and peak linked risk.	Workflow
case_transactions	Evidence links between cases and transactions.	Join
case_notes	Analyst notes with optional external evidence references.	Authored
case_events	The investigation timeline. Every mutation writes one row.	Append-only
audit_logs	Security-relevant actions with actor, IP, and before/after state.	Immutable
model_versions	Training provenance: date, record count, features, threshold, parameters.	One active
import_jobs	CSV upload outcomes with per-row error reports.	Tallied

AUDIT IMMUTABILITY — ENFORCED, NOT PROMISED

Migration 0001 installs two PostgreSQL rules that rewrite UPDATE and DELETE on audit_logs into no-ops. This was verified against the live database: an UPDATE , a targeted DELETE , and a bare DELETE FROM audit_logs executed as the table owner were all silently discarded, with the row count unchanged. Neither an application bug nor a compromised application role can rewrite history.

Least-privilege database role

The application connects as a dedicated sentinel role created with NOSUPERUSER , NOCREATEDB and NOCREATEROLE . The bootstrap script requires the superuser password exactly once, to create that role and the database, and never stores it. This was confirmed correct when the test suite — which had tried to create its own test database — was properly refused by PostgreSQL.

05 Risk Scoring Engine

Eight deterministic business rules plus one bounded machine learning factor.

Each rule fires only when its condition is met, and contributes a named, positive number of points. The analyst sees every rule that fired, its point value, and a plain-language explanation of the evidence.

RULE	CONDITION	MAX
AMOUNT_DEVIATION	Amount z-score against the customer's 90-day baseline, <i>or</i> plain multiple of their average — whichever is the stronger signal	28
NEW_DEVICE	Device never seen on this account (12 points if seen only once or twice)	20
NEW_LOCATION	City the customer has never transacted from, +5 beyond 500 km from their usual area	20
NEW_BENEFICIARY	First ever payment to this payee from this account	15
BEHAVIOR_CHANGE	Sustained drift in spend size and pace versus the established baseline	15
HIGH_FREQUENCY	≥3 transactions in 10 minutes, ≥6 within an hour, or ≥3× the daily norm	14
UNUSUAL_TIME	Hour holding under 5% of the customer's prior activity (needs ≥20 prior transactions)	12
AMOUNT_ABSOLUTE	Beyond the customer's own historical 99th percentile	10
ML_ANOMALY	Bounded uplift from the Isolation Forest, above its own decision boundary	15

Risk bands

BAND	SCORE	ANALYST GUIDANCE
LOW	0 – 29	Consistent with established behaviour. No review required.
MEDIUM	30 – 59	Some deviation from normal. Worth a look when capacity allows.
HIGH	60 – 79	Several behavioural signals triggered. Requires analyst review.
CRITICAL	80 – 100	Strong combination of signals. Prioritise for investigation.

The invariant the design protects

$$\text{business_score} == \sum (\text{points of the factors shown to the analyst})$$

Raw signal strength can exceed 100 when many rules fire together. Rather than clipping — which would leave the displayed factors summing to more than the displayed score — contributions are scaled proportionally onto the 0–100 scale using largest-remainder rounding, and the assessment records that scaling occurred. The explanation panel never shows numbers that fail to add up. This is asserted in the engine itself and verified by a randomised property test across thousands of generated feature combinations.

A worked example from the live system

FACTOR	POINTS	EVIDENCE SHOWN TO THE ANALYST
Unusual Amount	23	32.4× this customer's average of ₹8,735
New Device	17	Device not previously seen on this account
New Location	17	842 km from the customer's usual area, never used before
New Beneficiary	13	First transfer to this payee
ML Anomaly Signal	12	Isolation Forest anomaly score 1.00
Unusual Time	10	Occurred at 04:00, outside normal hours
High Value Transaction	8	Exceeds the customer's own 99th percentile
Total	100	CRITICAL — matches the displayed score exactly

WHY THE AMOUNT RULE TAKES THE STRONGER OF TWO MEASURES

A customer with near-constant spending has a standard deviation of zero, making the z-score undefined. Under a z-score-only rule, a 50× transaction on such an account would score nothing — precisely the case most worth catching. The rule therefore also evaluates a plain multiple of the customer's average and takes whichever signal is stronger.

06 Feature Engineering

24 documented features per transaction; 14 of them feed the model.

NO LOOKAHEAD

Every feature is computed strictly from the customer's history *preceding* the transaction being described. A score computed during a backfill is therefore identical to the score that transaction would have received live. The test suite asserts this directly: it recomputes features with later rows removed and requires every value to be unchanged.

FEATURE	DEFINITION	MODEL
<code>amount_zscore</code>	Standard deviations above the customer's prior mean; zero when history is too thin	Yes
<code>amount_vs_customer_avg_ratio</code>	This amount divided by the customer's prior average	Yes
<code>customer_avg_amount</code>	Expanding mean of prior amounts, excluding this transaction	—
<code>customer_std_amount</code>	Expanding standard deviation of prior amounts	—
<code>customer_p99_amount</code>	Prior 99th-percentile amount for this customer	—
<code>amount_value</code>	The raw transaction amount	—
<code>txn_count_10m</code>	Transactions in the 10 minutes up to and including this one	Yes
<code>txn_count_1h</code>	Transactions within the preceding hour	Yes
<code>txn_count_24h</code>	Transactions within the preceding 24 hours	Yes
<code>customer_avg_daily_txns</code>	Historical transactions per day, from prior activity only	—
<code>velocity_score</code>	Short-window rate relative to the customer's own norm	Yes
<code>hour_of_day</code>	Local hour, 0–23	Yes
<code>day_of_week</code>	Monday through Sunday as 0–6	Yes
<code>is_unusual_hour</code>	Hour holds under 5% of prior activity, with ≥ 20 prior transactions	—
<code>is_new_device</code>	Device unseen for this customer	—
<code>is_new_location</code>	City not in the customer's historical set	—
<code>is_new_beneficiary</code>	Payee never paid before	—
<code>device_usage_count</code>	Prior uses of this device by this customer	—
<code>distance_from_usual_km</code>	Haversine distance from the centroid of prior transaction locations	Yes
<code>unique_devices_30d</code>	Distinct devices used in the preceding 30 days	Yes
<code>unique_locations_30d</code>	Distinct cities transacted from in the preceding 30 days	Yes
<code>amount_trend_ratio</code>	7-day average amount over the longer-run average	Yes
<code>frequency_trend_ratio</code>	7-day transaction rate over the longer-run rate	Yes
<code>behavior_change_score</code>	Combined upward drift in spend and pace, 0 to 1	Yes

Computation strategy

Features are produced in two passes. A vectorised pandas pass handles statistics that expanding and rolling windows express directly. A single ordered pass per customer handles the stateful signals — novelty, 30-day distinct counts, the hour-of-day distribution and the location centroid — each of which requires memory of everything seen so far. Time-

window counts use NumPy `searchsorted` on pre-sorted timestamps, which is exact and behaves identically whether the frame holds one customer or ten thousand.

WHAT THE MODEL IS DELIBERATELY NOT SHOWN

Raw amounts, customer identifiers, and the boolean novelty flags are withheld from the model. It learns from behavioural shape; the novelty conditions are already covered explicitly and transparently by the business rules, where an analyst can see them.

07 Machine Learning Pipeline

Isolation Forest, kept as a separate and bounded signal.

```
load → clean → engineer features → scale → fit → score → normalise → combine with rules → persist
```

Live model

12,115

TRAINING RECORDS

14

INPUT FEATURES

200

ESTIMATORS

0.03

CONTAMINATION

Score normalisation

scikit-learn's `decision_function` is unbounded and centred on zero, which is impossible to compare across training runs. Output is mapped onto a 0–1 scale using percentile bounds captured at fit time and stored on the model version row. Reusing the training bounds is the point: a transaction scored next week lands on the same scale as one scored during training, so stored scores stay comparable and the threshold does not silently move.

The anomaly threshold is the normalised position of the model's own decision boundary, so "the model considers this anomalous" and "the score is at or above the threshold" are the same statement. On the live model that threshold is **0.7297**, with an observed anomaly rate of 3%.

How the model combines with the rules

The model contributes only above its own threshold, and its contribution is capped at 15 points. It appears in the explanation panel as one named factor alongside the business rules, with its own point value — so the final score remains fully attributable while still incorporating a signal the rules alone would miss.

GRACEFUL DEGRADATION

If no model has been trained, or its artifact cannot be loaded, the platform continues to operate. The business rules alone produce a complete and fully explainable score; the ML factor simply does not fire. The system never blocks on the model being available.

Detection performance on the live dataset

RISK BAND	TRANSACTIONS	SHARE
LOW	11,892	98.16%
MEDIUM	133	1.10%
HIGH	58	0.48%
CRITICAL	32	0.26%

08 On Model Metrics

What the system reports, and what it deliberately refuses to report.

STATED VERBATIM ON THE MODEL PERFORMANCE PAGE

"Model performance metrics requiring labeled ground truth are unavailable."

Precision, recall, F1 and confusion matrices require confirmed outcomes for each transaction — a verified record of which were genuinely fraudulent. This platform has no such labels. Any figure presented would therefore be fabricated, so the model page omits them entirely and explains why, rather than producing a number that looks rigorous and is not.

What is reported instead

REPORTED	BECAUSE
Model version, algorithm, training date	Provenance facts read from the registry
Training record count and feature list	Directly observed at fit time
Contamination, estimators, threshold	Configuration the model was fitted with
Observed anomaly rate	Measured proportion the fitted model marked anomalous
Analyst case outcomes	Real human decisions, shown on the analytics page

Analyst outcomes — the honest measure

The analytics page reports what analysts actually concluded. Alerts that nobody has judged yet are counted as **pending**, not folded into either bucket, because treating an unreviewed alert as a confirmation or a false positive would misrepresent the work done.

Alerts raised	90
Cases opened	28
Reached a terminal decision	12
Confirmed suspicious	3
False positive	6
Closed, no action	3
Still open	16
Pending review (no case yet)	62

Synthetic scenario coverage

Because the data generator records which transactions it injected, coverage against those scenarios can be measured. This is reported in a separately labelled panel marked **DEMO DATA ONLY — not a fraud-accuracy metric**. It describes generated data and says nothing about accuracy against real fraud.

INJECTED SCENARIO	GENERATED	REACHED REVIEW	REACHED HIGH RISK
Multiple anomalies combined	11	11 (100%)	11 (100%)
Large transaction spike	14	14 (100%)	11 (79%)
New device	14	14 (100%)	11 (79%)
Unusual location	17	16 (94%)	7 (41%)
Unusual transaction time	10	9 (90%)	6 (60%)
Transaction velocity spike	93	72 (77%)	44 (47%)

Velocity coverage is lower by design: the first transaction in a burst is not yet a burst, and correctly scores low at the moment it occurs.

09 Security

Controls implemented, and the reasoning behind each.

CONTROL	IMPLEMENTATION
Password hashing	bcrypt at cost 12. Passwords over 72 bytes are rejected rather than silently truncated, so two long passwords cannot collide.
Token strategy	Short-lived access token returned in the response body; refresh token in an httpOnly cookie. The access token is held in memory only — never <code>localStorage</code> , so an XSS payload cannot read it.
Token type confusion	A <code>type</code> claim is compared in constant time, so a refresh token cannot be replayed as an access token.
Algorithm downgrade	The decoder pins HS256; an <code>alg: none</code> token is rejected. Covered by test.
Immediate revocation	Every request re-reads the user and role from the database, so deactivation or demotion takes effect at once rather than at token expiry.
Brute-force protection	Sliding-window limiter keyed on IP <i>and</i> account, so an attacker cannot lock out a legitimate user by spraying their address.
Account enumeration	Identical message and timing for an unknown address and a wrong password — the hash comparison runs either way.
Authorisation	Enforced server-side on every endpoint. The UI mirrors the same matrix as a courtesy, never as the control.
SQL injection	SQLAlchemy ORM throughout; no query text is ever assembled from user input.
XSS	React's default escaping, plus a restrictive Content-Security-Policy and <code>nosniff</code> from the API.
CSV formula injection	Exported cells beginning <code>=</code> , <code>+</code> , <code>-</code> or <code>@</code> are neutralised so a spreadsheet treats them as text.
PII masking	Applied in the Pydantic response schemas. Source fields are excluded from serialisation, so no endpoint can leak them by omission.
Audit integrity	Append-only enforced by PostgreSQL rules, verified against the live database.
Secrets in logs	A redacting filter masks credential patterns, bearer tokens and bcrypt digests before any record is written.
Secrets in the audit trail	Known-sensitive keys are scrubbed before any state snapshot is persisted.
Error handling	Users receive a professional message and a stable code; stack traces and database detail go to the server log only.
Secrets in version control	<code>.env</code> gitignored and confirmed absent from the entire commit history before the repository was made public.

Data masking examples

FIELD	STORED	RETURNED BY THE API
Phone	9924xxxxxx	9924*****
Account number	xxxxxxxx2478	XXXX XXXX 2478
Email	krishna.joshi@...	k*****@e*****...
Name	Krishna Joshi	Krishna J.
Device fingerprint	DEV-8fa31c9e4b	DEV-****1c9e4b

Role-based access control

CAPABILITY	ANALYST	SENIOR	MANAGER	ADMIN
View dashboards, transactions, customers, alerts, cases	●	●	●	●
Create case, add note, link evidence, export	●	●	●	●
Assign, re-prioritise, escalate, resolve	—	●	●	●
Close and reopen cases, view audit log, run detection	—	—	●	●
Manage users, change roles, train model, import data	—	—	—	●
Total permissions	13	18	23	28

Verified against the live deployment: all four roles were signed in and every privileged endpoint was called by each, confirming both the grants and the refusals. Zero violations.

10 Application Features

Sixteen routes, every one reading live data.

MODULE	CAPABILITY
Login	Email and password with show/hide, remember-session, inline validation, loading and error states. Demo role hints are populated from non-secret environment configuration.
Risk Overview	Six KPI tiles, transaction volume over 24H/7D/30D/90D, risk distribution donut, suspicious activity trend with a high-risk sub-series, ranked detection reasons, risk by location, and a recent alerts feed.
Transaction Monitoring	Server-side filtering on eleven dimensions, debounced instant search, sortable amount / score / date / customer, 20-50-100 pagination, CSV and Excel export. Filters live in the URL so a view can be shared or bookmarked.
Transaction Investigation	Full summary, a visual risk gauge, the "Why was this flagged?" breakdown rendered from stored factors, the separated ML anomaly score, and the complete behavioural feature set behind the decision.
Customer Risk Profile	Masked identity, aggregate risk position, and a side-by-side contrast of established baseline against the last seven days with percentage deltas, plus amount and risk history charts.
Anomaly Detection	Worklist ranked by score, filterable to model-flagged transactions only, with on-demand re-running of detection across the dataset.
Alert Centre	Polling feed with an unread badge. Each alert can be viewed, investigated, assigned or dismissed, with permission-appropriate actions per role.
Investigation Cases	Full lifecycle — create, assign, re-prioritise, note, link and unlink evidence, escalate, resolve, mark false positive, close and reopen — each writing a timeline event.
Investigation Timeline	Read directly from the <code>case_events</code> table. Never reconstructed in the browser.
Analytics	Volume, average risk trend, high-risk trend, detection reason analysis, value by risk band, payment method breakdown, and false-positive analysis from real case outcomes.
Reports	Risk summary for any period with distribution, leading signals, top locations and the highest-risk transactions, exportable to CSV and Excel.
Model Performance	Version, training provenance, configuration, the full feature dictionary, version history, scenario coverage, and the explicit statement on unavailable metrics.
Data Ingestion	CSV upload with drag-and-drop, per-row validation, and a summary reporting records received, valid, invalid, duplicate, processed and flagged, with each rejected row explained individually.
Audit Log	Searchable, filterable, read-only view of the append-only trail with actor, action, resource, IP and before/after state.
User Management	Create users, change roles, deactivate and reactivate, with guards preventing removal of the last administrator or self-deactivation. Shows the live permission matrix.
System Settings	Effective configuration, risk thresholds, model parameters and security settings, plus self-service password change. No secret is exposed.

CSV import validation

A file with three bad rows imports the rest and reports exactly which three failed and why, rather than rejecting the upload wholesale or silently skipping rows. Demonstrated on the live system with a deliberately malformed file:

ROW	FIELD	REASON
4	customer_id	Unknown customer reference CUST-NOPE
5	amount	Amount must be greater than zero
6	timestamp	Timestamp is not a recognised format
7	transaction_id	Duplicate transaction ID within the file

Result: 6 received · 2 valid · 3 invalid · 1 duplicate · 2 processed and scored.

11 API Surface

48 operations, documented automatically through OpenAPI.

Every endpoint except `/health` and `/api/auth/login` requires authentication. This was verified programmatically: all 45 protected operations were called without a token and every one returned `401`.

GROUP	ENDPOINTS
Authentication	<code>POST /auth/login</code> · <code>POST /auth/refresh</code> · <code>POST /auth/logout</code> · <code>GET /auth/me</code>
Dashboard	<code>GET /dashboard/summary</code>
Transactions	<code>GET /transactions</code> · <code>GET /transactions/{ref}</code> · <code>PATCH /transactions/{ref}/status</code> · <code>GET /transactions/export</code> · <code>GET /transactions/locations</code>
Customers	<code>GET /customers</code> · <code>GET /customers/{ref}</code> · <code>GET /customers/{ref}/risk</code>
Alerts	<code>GET /alerts</code> · <code>POST /alerts/{ref}/acknowledge</code> · <code>POST /alerts/{ref}/assign</code> · <code>POST /alerts/{ref}/dismiss</code>
Cases	<code>GET /cases</code> · <code>POST /cases</code> · <code>GET /cases/{ref}</code> · <code>PATCH /cases/{ref}</code> · <code>POST /cases/{ref}/notes</code> · <code>POST /cases/{ref}/transactions</code> · <code>DELETE /cases/{ref}/transactions/{id}</code> · <code>GET /cases/{ref}/timeline</code>
Analytics & reports	<code>GET /analytics</code> · <code>GET /analytics/outcomes</code> · <code>GET /reports/risk-summary</code> · <code>GET /reports/risk-summary/export</code>
Detection & model	<code>GET /model/status</code> · <code>POST /model/train</code> · <code>POST /detection/run</code> · <code>GET /anomalies</code>
Ingestion	<code>POST /transactions/import</code> · <code>GET /imports</code> · <code>GET /imports/{id}</code> · <code>GET /transactions/import/template</code>
Administration	<code>GET /audit-logs</code> · <code>GET /audit-logs/actions</code> · <code>GET /users</code> · <code>POST /users</code> · <code>PATCH /users/{id}</code> · <code>GET /users/assignable</code> · <code>POST /users/me/password</code> · <code>GET /settings</code> · <code>GET /settings/roles</code>
Search & system	<code>GET /search</code> · <code>GET /health</code>

Error contract

Every failure returns the same envelope, with a stable machine-readable code and a message safe to show a user:

```
{ "error": { "code": "NOT_FOUND", "message": "The requested record could not be found." } }
```

Server-side faults additionally carry a trace identifier so a user report can be correlated with the server log, while the log itself holds the stack trace that the response never exposes.

12 Testing & Verification

265 tests, all passing.

265 TESTS PASSING	205 INFRASTRUCTURE-FREE	60 API & PERSISTENCE	6 REAL BUGS CAUGHT
----------------------	----------------------------	-------------------------	-----------------------

AREA	WHAT IS COVERED
Risk scoring	The score/factor invariant including a randomised fuzz, band boundaries, every individual rule, the zero-variance fallback, and the ML contribution cap
Feature engineering	Output shape, the no-lookahead property, cold-start suppression, novelty detection, velocity windows, and customer isolation
Machine learning	Training, scoring, determinism, persistence round-trip, normalisation monotonicity, threshold agreement, and guardrails
Security	Hashing, token forgery, type confusion, <code>alg: none</code> downgrade, rate limiting, masking, log redaction, audit scrubbing
Permissions	The full matrix, plus live RBAC denials for every role against every privileged endpoint
Import validation	Timestamp formats, amount ranges, encoding, missing columns, oversized files
API and workflow	Authentication, session handling, case lifecycle, timeline persistence, audit writes, PII masking in responses

Defects the tests caught during development

These were found by the suite and by verification against a live database, and all were fixed:

DEFECT	CONSEQUENCE HAD IT SHIPPED
Lookahead leak	<code>customer_avg_daily_txns</code> was computed over each customer's entire history including future transactions, so a backfilled score would not have matched the score the transaction received live.
Raw PII in responses	Customer schemas serialised <code>full_name</code> , <code>email</code> , <code>phone</code> and <code>account_number</code> alongside their masked versions, defeating the entire masking layer.
Log redaction ordering	The key/value rule ran before the bearer rule, so <code>Authorization: Bearer <token></code> masked the word "Bearer" and left the token exposed in logs.
Invalid SQL cast	Casting a boolean to float for conditional counts is rejected by PostgreSQL, returning <code>500</code> on the customer list for every role.
Stale ORM relationship	After assigning a case the identity map returned a cached assignee, so a just-assigned case still displayed as unassigned.
Missing timeline event	Evidence linked while opening a case wrote no <code>TRANSACTION_LINKED</code> event — and that is the primary flow, so those cases showed a timeline with no record of the evidence being attached.

Verification against the live deployment

CHECK	RESULT
Login, all four roles	Correct permission counts of 13 / 18 / 23 / 28
Session cookie	<code>HttpOnly; SameSite=None; Secure</code>
RBAC matrix	Zero violations, including every expected refusal
Explainability	Factors summed exactly to the score on every high-risk transaction sampled
PII masking	No raw identifier present in any response
Audit immutability	<code>UPDATE</code> , <code>DELETE</code> and bare <code>DELETE FROM</code> all discarded
SPA deep links	All routes resolve; no 404 on refresh
Production hardening	<code>/docs</code> disabled, all security headers present

13 Deployment

Three managed services, all free-tier.

COMPONENT	PLATFORM	CONFIGURATION
Database	Supabase (PostgreSQL 17)	Mumbai region, session pooler, TLS required. 23 MB of the 500 MB allowance.
API	Render web service	Singapore region, Python 3.11, deployed from a render.yaml Blueprint.
Frontend	Render static site	Global CDN, SPA rewrite, immutable caching on hashed assets.

Cross-origin considerations

The three services sit on different origins, which changes assumptions that held on localhost. The session cookie is issued `SameSite=None; Secure` so it can travel between the frontend and the API; because credentials are involved, a wildcard CORS origin is not permitted, so the API is given the frontend's exact origin.

Operational notes

- **Cold starts.** The free API instance sleeps after 15 minutes idle; the next request takes roughly 60 seconds. A scheduled health-check ping removes this entirely.
- **Model artifact.** Render's filesystem is ephemeral, so the trained artifact travels with the repository and the registry stores only a filename, resolved against the artifacts directory wherever it runs.
- **Migrations and seeding.** Run against the managed database from a developer machine, since the free plan provides no remote shell.

Project metrics

MEASURE	VALUE
Backend application code	10,073 lines across 66 files
Frontend application code	10,019 lines across 33 files
Tests	2,100 lines across 9 files
Scripts and migrations	1,672 lines
Total	24,061 lines · 141 files
API operations	48
Database tables / indexes / foreign keys	16 / 63 / 25
Engineered features (model inputs)	24 (14)
Scoring rules	8 + 1 bounded ML factor
Roles / permissions / audit action types	4 / 28 / 23

Deliberate scope decisions

The following were considered and consciously excluded, and are recorded as such in the repository rather than left as apparent oversights:

- **Docker and CI.** Three managed free tiers carry the stack; container orchestration would add weight before there is anything to orchestrate.
- **WebSockets.** The alert centre polls on an interval. For an internal console this is far simpler to make correct and survives reconnects, sleeping laptops and proxy timeouts without dedicated reconnect logic.
- **Redis and Celery.** The deployment runs a single API process, so the in-memory rate limiter suffices. Its interface allows a shared store to be substituted without changing callers.
- **PDF report generation.** Reports render on screen and export to CSV and Excel.

CLOSING NOTE

The system's most distinctive property is what it declines to claim. It surfaces transactions for review, shows precisely why, and refuses to present an anomaly as a finding of fraud or to publish accuracy metrics it cannot substantiate. In a domain where a false accusation carries real cost to a real customer, that restraint is the feature.